The Plinkus Implementation Manual

Part II — From a Tree to a Running Virtual Machine

Abstract Syntax Trees · Recursive-Descent Parsing · Bytecode & Opcodes · The Compiler · The Stack VM

A self-contained field guide for the next four stages of your interpreter, written to be read cover to cover with nothing but pen, paper, and a Python interpreter. The lexer is behind you; everything ahead is in these pages.

How to use this manual

Read straight through. When you reach an **Exercise**, stop and work it out before turning the page — the answer is deliberately hidden on the *following* page so you can commit to an attempt first. Exercises marked as runnable are meant to be typed into the Python interpreter; the rest are pen-and-paper. There are no drawn diagrams anywhere in this manual: every tree, stack, and byte stream is written out as indented or parenthesised text, which you can reproduce on paper exactly as printed.

Pipeline status: Lexer ✓ **done** → AST → Parser → Opcodes → Compiler → VM

What is in this manual

This is Part II. It picks up exactly where your lexer left off and carries you through the remaining four stages of the pipeline, in build order. Each part is written so that you could implement that stage the moment you finish reading it.

- **Part 0 — The lay of the land.** Where the AST sits, and the whole road ahead in one mental model.
- **Part 1 — The Abstract Syntax Tree.** What it is, why the token list is not enough, how to design node types for Plinkus, and how to represent them in Python with no libraries.
- **Part 2 — The Parser.** Recursive descent, the precedence cascade, associativity, statements, and fail-fast errors. This is the hardest part; it gets the most pages.
- **Part 3 — Opcodes & Bytecode.** The stack machine model, designing the Plinkus instruction set, and how a chunk is laid out.
- **Part 4 — The Compiler.** Walking the tree to emit bytecode, post-order emission, globals, and jump backpatching.
- **Part 5 — The Virtual Machine.** The fetch–decode–execute loop, the value stack, runtime types, and a full end-to-end trace.
- **Part 6 — A plan of attack.** Milestones, the bugs that will bite you, and what to design on paper versus in code.

> **A note on scope, so you do not over-build**
>
> Your locked-in feature set — `let`, `print`, integer/float/string literals, the full comparison set, `if/else` with braces, arithmetic with int→float promotion, and no logical operators — is exactly the right size for this manual. Everything here is written against that set. Where a stretch feature (loops, blocks with scope) changes the picture, it is flagged, but the core path never assumes it.

PART 0

The Lay of the Land

One mental model to hold everything else together.

Before the details, fix the shape of the whole machine in your head. Your interpreter is a *pipeline*: source text goes in one end, and observable behaviour (numbers printed to the screen) comes out the other. Each stage consumes the output of the previous stage and produces something more structured — or, in the last two stages, something more primitive.

Here is the entire journey of a single line of Plinkus, written as a chain. Keep coming back to this list; every part of this manual is really just a close-up of one arrow in it.

```
SOURCE      let total = 2 + 3 * 4;
   |  (lexer  -- done!)
TOKENS      LET  IDENT(total)  EQUAL  INT(2)  PLUS  INT(3)
            STAR  INT(4)  SEMICOLON  EOF
   |  (parser)
AST         LetStmt(name='total',
                    value=Binary('+', Literal(2),
                                      Binary('*', Literal(3), Literal(4))))
   |  (compiler)
BYTECODE    OP_CONSTANT 0     ; push 2
            OP_CONSTANT 1     ; push 3
            OP_CONSTANT 2     ; push 4
            OP_MUL            ; 3*4 -&gt; 12
            OP_ADD            ; 2+12 -&gt; 14
            OP_DEFINE_GLOBAL 3; bind 'total' = 14
   |  (virtual machine)
EFFECT      the global 'total' now holds 14
```

Notice the pattern. The lexer turned *characters* into *tokens* — it removed the noise of whitespace and grouped digits into numbers, but it produced a flat list. The parser is about to turn that flat list into a *tree* that captures how the pieces nest. The compiler will flatten the tree back into a flat list — but a very different one, a list of simple instructions for a machine. And the VM will run those instructions.

Why go up to a tree and then back down to a flat list? Because the tree is where *meaning* lives. The token list `2 + 3 * 4` does not, by itself, say whether you multiply first or add first. The tree does: it physically puts the multiplication *below* the addition, so whoever walks the tree is forced to evaluate it first. Structure encodes precedence. That single idea is the reason the AST exists, and it is the spine of the next forty pages.

The one-sentence summary of each remaining stage

- **AST:** a data structure of nested Python objects that captures how the program is grouped.
- **Parser:** the code that reads tokens left to right and builds that data structure, respecting precedence.
- **Opcodes:** your hand-designed vocabulary of tiny instructions (push a constant, add, print, jump).
- **Compiler:** a walk over the AST that emits a list of those instructions.
- **VM:** a loop that reads the instructions one by one and does what each says, using a stack.

PART 1

The Abstract Syntax Tree

The data structure that holds the shape of a program.

1.1 Why the token list is not enough

Your lexer produces a stream like `[INT(2), PLUS, INT(3), STAR, INT(4)]`. This is a *sequence*: it knows the order of the symbols but nothing about their grouping. Consider what the same sequence would mean under two different groupings. If you read it as "(2 + 3) then times 4" you get 20. If you read it as "2 plus (3 times 4)" you get 14. Both are consistent with the flat list. The list alone is ambiguous; the grouping is the missing information, and 14 is the answer we want because multiplication binds tighter than addition.

An Abstract Syntax Tree is a tree of objects that records exactly one grouping. Once the tree exists, the ambiguity is gone: there is one root, and every operator sits above the sub-expressions it operates on. Whoever processes the tree simply obeys its shape.

Reading a tree written as text

This manual never draws trees; it writes them. There are two textual notations you will use constantly, and both are worth internalising now.

Parenthesised (S-expression) form. Put the operator first, then its operands in brackets. The expression 2 + 3 × 4, grouped correctly, is:

```
(+ 2 (* 3 4))
```

Read it inside-out: the innermost complete bracket, `(* 3 4)`, must be evaluated before the `+` can run, because it is *nested inside* the plus. Nesting is precedence.

Indented outline form. The same tree, one node per line, with indentation showing parent-child depth:

```
Binary '+'
    left:  Literal 2
    right: Binary '*'
               left:  Literal 3
               right: Literal 4
```

The root is the `+` node; it has two children. Its left child is a leaf holding 2. Its right child is another operator node, which has its own two leaf children. Leaves (the literals) are the values; internal nodes are the operations. Evaluating a tree means evaluating the children first, then combining — which naturally happens bottom-up.

The way a parent node interacts with its children is the whole point, so make it concrete: think of the tree as a recipe with sub-recipes. The `+` node cannot do its job until both of its ingredients are ready, and one of those ingredients — the `*` node — is itself a sub-recipe that must be finished first. So you cook from the leaves upward: each operator node waits for its children to hand up finished values, combines them into one value, and hands *that* up to *its* parent. A value only ever flows one way — upward, from child to parent — and an operator never sees raw sub-expressions, only the single finished values its children produce. That child-hands-value-to-parent contract is exactly what the compiler and the VM will act out later, one as emitted instructions and one as pushes on a stack.

Exercise 1 — Grouping by hand (paper)

For the expression below, write the AST in **both** parenthesised form and indented outline form, respecting normal arithmetic precedence (multiplication and division before addition and subtraction).

```
3 + 4 * 5 - 6
```

Then state, in one sentence, which operation the root node holds and why.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 1

Precedence first: the two multiplications/divisions bind before the additive operators, so `4 * 5` is one unit. Then the additive operators are **left-associative**: `3 + (4*5) - 6` groups as `((3 + (4*5)) - 6)`. Parenthesised form:

```
(- (+ 3 (* 4 5)) 6)
```

Indented outline form:

```
Binary '-'
    left:  Binary '+'
               left:  Literal 3
               right: Binary '*'
                          left:  Literal 4
                          right: Literal 5
    right: Literal 6
```

The root is the subtraction. Because the additive operators are left-associative and the subtraction is the last one applied, it ends up on the outside — the root is always the operation performed *last*, i.e. the one that combines the two largest sub-results.

1.2 Concrete syntax versus abstract syntax

The word *abstract* in AST is doing real work. A *concrete* syntax tree (also called a parse tree) records every syntactic detail: every parenthesis you typed, every semicolon, every grammar rule the parser stepped through. An abstract syntax tree throws away everything that was only there to guide grouping, and keeps only what affects meaning.

Example: the user writes `(2 + 3)`. The parentheses matter while parsing — they force the addition to group first — but once the tree is built, the grouping is already captured by the tree's shape. So the AST for `(2 + 3)` and for `2 + 3` (in a context where they group the same way) is *identical*: a single Binary '+' node over two literals. There is no “parenthesis node.” The parentheses did their job and vanished. Likewise the semicolon that ends a statement is a signal to the parser, not a node in the tree.

> **Practical consequence for Plinkus**
>
> You do **not** need node types for parentheses, semicolons, or braces. Braces and semicolons are consumed by the parser as it recognises structure; parentheses only influence which node becomes a child of which. Your node catalogue will be pleasantly small.

1.3 Designing the Plinkus node types

There are two families of nodes, and the split matters because they behave differently: **expressions** produce a value; **statements** perform an action and produce no value. `2 + 3` is an expression — it stands for the value 5. `print(x);` is a statement — it does something (writes to the screen) and does not itself stand for a value. A Plinkus program is, at the top level, a list of statements.

The two families are not separate worlds; they plug into each other in one specific direction, and knowing that direction tells you how the tree fits together. **Statements contain expressions, never the reverse.** A `print` statement holds an expression whose value it will print; a `let` statement holds an expression whose value it will bind to a name; an `if` statement holds an expression it will test. So an expression node is always

found *dangling off* a statement node (or off another expression), as a child, supplying it a value. This is like a recipe step (a statement: “add the sauce”) that refers to an ingredient (an expression: “200ml of the reduced stock”): the step is the action, the ingredient is the thing with a value the action consumes. When you build the tree, that is the join you are looking for — every statement reaches down into one or more expressions for its values.

The expression nodes

Derived directly from your feature set, you need exactly these:

- **Literal** — a constant value already known at parse time. Fields: `value` (a Python int, float, or str) and, usefully, the source `line`. Covers your int/float/str literals.
- **Variable** — a use of a name, e.g. the `x` in `x + 1`. Fields: `name`, `line`. This means “look up the current value bound to this name.”
- **Binary** — an operation with a left and right operand. Fields: `op` (which operator), `left`, `right`, `line`. Covers `+ - * /` and all six comparisons.
- **Unary** — a one-operand operation. Fields: `op`, `operand`, `line`. You need this for negation, e.g. `-x` and `-5`. (You will not need logical NOT, since logical operators are out of scope.)

That is the complete expression set. Grouping (parentheses) does not appear — as established, it is absorbed into tree shape. Some implementations add a Grouping node so the AST can be printed back exactly; you do not need it, and leaving it out keeps the compiler simpler.

The statement nodes

- **PrintStmt** — field: `expr` (the expression to evaluate and print).
- **LetStmt** — a declaration. Fields: `name` (the variable being declared) and `value` (the expression whose result it is bound to).
- **ExprStmt** — an expression used as a statement, e.g. a bare expression followed by a semicolon. Field: `expr`. Its value is computed and then discarded. You may not need this for your two example programs, but it is the natural home for “an expression on its own line,” and it is one line to add.
- **IfStmt** — fields: `condition` (an expression), `then_branch` (a statement or block), and `else_branch` (a statement or block, or `None` if absent).
- **Block** — the thing inside a pair of braces. Field: `statements` (a Python list of statement nodes). Even if you do not implement local scope yet, a Block is just “run these statements in order,” and your `if` branches are blocks.

Exercise 2 — Deriving nodes from your own examples (paper)

Take this small Plinkus program and write the full AST for it in indented outline form. Use the node names from section 1.3. Assume `if` has no else branch here.

```
let n = 3 * 2;
if (n &gt; 5) {
    print(n);
}
```

Remember: the whole program is a *list* of statements at the top level.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 2

Two top-level statements. The first is a declaration whose value is a Binary; the second is an If whose condition is a comparison Binary and whose then-branch is a Block containing one Print.

```
Program (list of 2 statements)
  [0] LetStmt name='n'
          value: Binary '*'
                     left:  Literal 3
                     right: Literal 2
  [1] IfStmt
          condition: Binary '&gt;'
                         left:  Variable 'n'
                         right: Literal 5
          then_branch: Block
              [0] PrintStmt
                       expr: Variable 'n'
          else_branch: None
```

Points to notice: `n` inside the condition and inside the print is a **Variable** node (a *use*), whereas the `n` in `let n` is just a string field on the LetStmt (a *declaration*), not a Variable node. That asymmetry trips people up: declaring a name and using a name are different acts, and only the use becomes a Variable node.

1.4 Representing nodes in Python without libraries

You want each node to be a small object with named fields. “No external libraries” does not rule out the standard library, and the standard library gives you the cleanest option: `dataclasses`. A dataclass writes the boilerplate constructor for you and gives you a readable `repr` for free, which is priceless when debugging a tree. Here is the whole expression hierarchy:

```
from dataclasses import dataclass

# --- expressions ---
@dataclass
class Literal:
    value: object      # int, float, or str
    line: int

@dataclass
class Variable:
    name: str
    line: int

@dataclass
class Unary:
    op: str            # e.g. '-'
    operand: object    # an expression node
    line: int

@dataclass
class Binary:
    op: str            # e.g. '+', '&lt;=', '=='
    left: object       # an expression node
    right: object      # an expression node
    line: int
```

And the statements follow the same pattern:

```
# --- statements ---
@dataclass
```

```
class PrintStmt:
    expr: object
    line: int

@dataclass
class LetStmt:
    name: str
    value: object
    line: int

@dataclass
class ExprStmt:
    expr: object
    line: int

@dataclass
class Block:
    statements: list
    line: int

@dataclass
class IfStmt:
    condition: object
    then_branch: object   # usually a Block
    else_branch: object   # a Block or None
    line: int
```

Why dataclasses and not something fancier

You will see the *Visitor pattern* mentioned everywhere in compiler texts, and the *Crafting Interpreters* book generates its node classes with a script. In Python, with your scope, that machinery is overkill. Plain dataclasses plus a walk based on `isinstance` checks (shown in section 1.6) is idiomatic, short, and easy to debug. Do not let the fancy names intimidate you into over-engineering — the fail-fast, keep-it-small philosophy that served your lexer applies here too.

Carrying a `line` field on every node looks tedious, but it is the single best decision you can make for your fail-fast error philosophy. When the compiler or VM hits a problem, it can say `Line 7: cannot add int and str` because the node remembers where it came from. Populate it from the token you were looking at when you built the node.

Exercise 3 — Build a tree by hand in Python (runnable)

Type the dataclass definitions for `Literal` and `Binary` into the interpreter. Then **construct, by hand,** the AST object for the expression `2 + 3 * 4` (correctly grouped) and print it. You should see the nesting reflected in the repr.

You are not writing a parser here — you are assembling the objects manually, the way the parser eventually will.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 3

The multiplication is the right child of the addition:

```
&gt;&gt;&gt; from dataclasses import dataclass
&gt;&gt;&gt; @dataclass
... class Literal:
...     value: object
...     line: int
&gt;&gt;&gt; @dataclass
... class Binary:
...     op: str
...     left: object
...     right: object
...     line: int
&gt;&gt;&gt; tree = Binary('+',
...     Literal(2, 1),
...     Binary('*', Literal(3, 1), Literal(4, 1), 1),
...     1)
&gt;&gt;&gt; print(tree)
Binary(op='+', left=Literal(value=2, line=1),
       right=Binary(op='*', left=Literal(value=3, line=1),
                    right=Literal(value=4, line=1), line=1), line=1)
```

The free `repr` from the dataclass shows you the whole tree. If you had grouped it wrong — putting the plus inside the times — the nesting in the printout would be visibly different, which is exactly how you will sanity-check your parser later.

1.5 How the tree encodes precedence and associativity

It is worth stating the invariant precisely, because your parser's whole job is to preserve it: **a node deeper in the tree is evaluated before a node above it.** So if `*` must happen before `+`, the multiplication node must be a *descendant* of the addition node, never the reverse.

Associativity is the same idea applied to operators of equal precedence. Subtraction is left-associative: `10 - 3 - 2` means `(10 - 3) - 2 = 5`, not `10 - (3 - 2) = 9`. In tree terms, the *left* operand of the outer subtraction is itself a subtraction. Left operand nested, right operand a leaf: that is what left-associative looks like as a shape. Right-associative operators (which you do not have, but assignment often is one) nest the other way.

> **Exercise 4 — An AST printer (runnable)**
>
> Write a function `show(node)` that returns the parenthesised (S-expression) form of an expression tree built from `Literal` and `Binary` nodes. For a Literal it returns the value as a string; for a Binary it returns `(op left right)` with the children recursively shown.
>
> Test it on your tree from Exercise 3 — you should get `(+ 2 (* 3 4))`. This function is the single most useful debugging tool you will build; keep it.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 4

The structure of the function mirrors the structure of the tree — one branch of the `if` per node type. This recursive shape ("dispatch on node type, recurse into children") is the template for the compiler too, so it is worth getting comfortable with it now.

```
def show(node):
    if isinstance(node, Literal):
        return str(node.value)
    if isinstance(node, Binary):
        return f"({node.op} {show(node.left)} {show(node.right)})"
    raise Exception(f"show: unknown node {node!r}")

&gt;&gt;&gt; show(tree)
'(+ 2 (* 3 4))'
```

Extend it as you add node types (a `Variable` returns its name; a `Unary` returns `(op operand)`). When your parser is running, feed its output through `show` and eyeball the grouping — it turns an invisible bug ("why is 2+3*4 giving me 20?") into a visible one ("oh, the printer says `(* (+ 2 3) 4)`, my precedence is inverted").

1.6 Walking the tree

Every later stage that consumes the AST — the printer above, the compiler soon — is a *tree walk*: a function that looks at a node, does something appropriate for its type, and recurses into the child nodes. You have two idiomatic ways to write the dispatch in Python.

Option A — a chain of `isinstance` checks, as in `show` above. Simple, all logic in one place, perfectly fine for a project this size.

Option B — dispatch by type name. A dictionary mapping node type to a handler function, or a class with one method per node type. This scales better if you have many node types, and it is the essence of the Visitor pattern dressed in Python clothes:

```
class Compiler:
    def compile(self, node):
        method = getattr(self, 'c_' + type(node).__name__, None)
        if method is None:
            raise Exception(f"no compiler for {type(node).__name__}")
        return method(node)

    def c_Literal(self, node): ...
    def c_Binary(self, node):  ...
    def c_PrintStmt(self, node): ...
```

Both are correct. For your IA, Option A is easier to explain in your writeup and easier to debug; Option B earns its keep only once the node count grows. Pick one and be consistent. The important idea, either way, is that a tree walk is defined *by recursion following the tree's shape* — you never write a loop over a flat list; you let the call stack mirror the tree.

The mental bridge to the parser

You just wrote functions that *consume* a tree by recursing into children. The parser does the mirror image: it *produces* a tree by recursing into sub-expressions. If you understood why `show` calls itself on `node.left` and `node.right`, you already understand the core of recursive descent — you will just be building the children instead of reading them.

PART 2

The Parser

Turning the flat token stream into the tree. The hardest stage — read slowly.

The parser reads the token list your lexer produced, left to right, exactly once, and builds the AST. The technique you will use is **recursive descent**, and it is called that because it is a set of mutually recursive functions that descend through the grammar rules from the loosest-binding construct to the tightest. It is the most widely used hand-written parsing technique in real compilers precisely because it is understandable and debuggable — which is exactly what you want.

2.1 Grammar: the specification you are implementing

Before writing a parser you write a *grammar*: a set of rules that says what a valid program looks like. The standard notation is EBNF. Each rule names a construct and lists what it is made of. Read `->` as "is made of," `|` as "or," `*` as "zero or more," and `?` as "optional." Here is a grammar for the Plinkus core:

```
program     -&gt; statement* EOF ;

statement   -&gt; printStmt
             | letStmt
             | ifStmt
             | block
             | exprStmt ;

printStmt   -&gt; "print" "(" expression ")" ";" ;
letStmt     -&gt; "let" IDENTIFIER "=" expression ";" ;
ifStmt      -&gt; "if" "(" expression ")" block ( "else" block )? ;
block       -&gt; "{" statement* "}" ;
exprStmt    -&gt; expression ";" ;
```

And now the part that carries all the precedence — the expression rules. The trick is one rule per precedence level, each rule referring *down* to the next-tighter level. Order them from loosest to tightest:

```
expression  -&gt; equality ;
equality    -&gt; comparison ( ( "==" | "!=" ) comparison )* ;
comparison  -&gt; term ( ( "&lt;" | "&gt;" | "&lt;=" | "&gt;=" ) term )* ;
term        -&gt; factor ( ( "+" | "-" ) factor )* ;
factor      -&gt; unary  ( ( "*" | "/" ) unary )* ;
unary       -&gt; "-" unary | primary ;
primary     -&gt; INT | FLOAT | STRING | IDENTIFIER
             | "(" expression ")" ;
```

Stare at this ladder until it clicks, because it *is* the parser. The loosest operators (equality) are at the top; the tightest binding thing (a literal, a name, or a parenthesised group) is at the bottom. Each level says "parse one of the next level down, then, as long as you keep seeing my operators, keep parsing more of the next level down and combining." Because `factor` (which handles `*`) sits *below* `term` (which handles `+`), a multiplication is always gathered into a single unit before the surrounding addition sees it — and that is how the grammar itself enforces precedence.

To see how the levels physically interact, picture a stack of graduated sieves with the coarsest mesh on top and the finest at the bottom. Pouring the tokens in, the top sieve (equality) lets everything fall straight through *except* the `==` and `!=` it is responsible for; the next sieve catches the comparisons, and so on down, until the finest sieve (primary) catches the indivisible atoms — a literal or a name. In code terms, each level does not itself parse the tighter operators; it *delegates* by calling the level below and waits to receive a

finished sub-tree back, then only wraps that sub-tree if it sees one of its own operators. So `term`, wanting an operand for a `+`, calls `factor`; `factor` quietly swallows any `*` or `/` it finds and returns a single completed node; and `term` receives that node already knowing nothing of the multiplication inside it. Each level trusts the level below to hand back one finished value, exactly the way each tree node trusts its children to hand up finished values in section 1.1. The call chain going down and the finished sub-trees coming back up are the two directions of that single handshake.

The reflex that makes this easy to remember

- **Lower in the ladder = binds tighter = deeper in the tree.**
- Each rule *starts* by calling the level below it, then *loops* on its own operators.
- The loop is what makes operators left-associative — you will see why in section 2.4.

Exercise 5 — Why left, not right (paper)

Using the `term` rule `term -> factor ( ("+"|"-") factor )*`, explain in two or three sentences why the tree for `10 - 3 - 2` must come out as `((10 - 3) - 2)` and not `(10 - (3 - 2))`.

Hint: think about what the loop has already built by the time it reaches the second minus.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 5

The rule parses the first `factor` (10), then enters the loop. First iteration: it sees `-`, parses the next factor (3), and combines them into `Binary('-', 10, 3)`. Crucially, it stores that combined node back as the "left so far." Second iteration: it sees the next `-`, parses 2, and combines the *already-built* `(10 - 3)` node with 2, giving `Binary('-', (10-3), 2)`.

So the left operand of the outer minus is the previous result and the right operand is always a single fresh factor. The accumulation happens on the left, which *is* left-associativity. To get right-associativity you would instead recurse on the right (call the same level again for the right operand) rather than loop — but you have no right-associative operators, so the loop is exactly what you want.

2.2 The parser's state and its helper methods

Just as your lexer had a `start`/munch/slice skeleton over the source string, the parser has a tiny skeleton over the token list. It holds the token list and a cursor `current` into it, and offers a handful of helpers that everything else is built from. Learn these five; the entire parser is written in terms of them.

The cursor is the piece that ties the helpers together, so be clear on how they interact with it. Think of `current` as a read-head resting on one token of the list, the way a bookmark rests between pages. `peek` *looks* at the token under the head without moving it; `advance` takes the token under the head and then *slides the head one step right*. Every other helper is built from those two: `check` is a peek that compares the type but does not move; `match` is a check that, *only if it succeeds*, calls advance to slide forward; `consume` is a check that either slides forward or aborts the whole parse. The single rule that keeps the parser sane is that the head moves in exactly one place — inside `advance` — so every step forward through the tokens is one call to it, and nothing ever quietly skips or re-reads a token behind the others' backs.

```
class Parser:
    def __init__(self, tokens):
        self.tokens = tokens
        self.current = 0

    def peek(self):            # the token we are about to consume
        return self.tokens[self.current]

    def check(self, kind):     # is the next token of this type?
        return self.peek().kind == kind

    def advance(self):         # consume and return the current token
        tok = self.tokens[self.current]
        if tok.kind != 'EOF':
            self.current += 1
        return tok

    def match(self, *kinds):   # if next is one of kinds, consume it
        if self.peek().kind in kinds:
            self.advance()
            return True
        return False

    def consume(self, kind, message):   # demand a token or fail fast
        if self.check(kind):
            return self.advance()
        tok = self.peek()
        raise SyntaxError(f"Line {tok.line}: {message}")
```

Notice how naturally your **fail-fast philosophy** lives in `consume`. When the parser *expects* a specific token — a closing parenthesis, a semicolon — and it is not there, there is no recovery: it raises a `SyntaxError` with the line and a human message, and the whole parse stops. This is the same "Super Saiyan punishment" you used in the lexer, and it keeps the parser dramatically simpler than any version that tries to recover and continue.

The difference between `match` and `consume` is worth pinning down. `match` is for *optional* tokens: "if the next token is a plus, take it and tell me so; otherwise leave it and tell me no." `consume` is for *required* tokens: "the next token had better be a right-paren, or the program is malformed." Loops use `match`; structural punctuation uses `consume`.

> **Exercise 6 — Implement the helpers (runnable-ish)**
>
> Assume a token is any object with `.kind` and `.line` attributes. Write `match(self, *kinds)` and `consume(self, kind, message)` from scratch (do not just copy them from memory of the text above — reconstruct them). Then, in words, describe what goes wrong if `consume` forgot to actually advance past the token it accepted.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 6

```
def match(self, *kinds):
    if self.peek().kind in kinds:
        self.advance()
        return True
    return False

def consume(self, kind, message):
    if self.check(kind):
        return self.advance()
    tok = self.peek()
    raise SyntaxError(f"Line {tok.line}: {message}")
```

If `consume` did not advance: it would verify the token is correct and return it, but leave the cursor sitting on that same token. The caller would proceed believing it had moved past, e.g. past a `)`, but the next helper would see the `)` again. In practice you would get an infinite loop or a cascade of bizarre "expected X" errors, because the parser is stuck re-reading one token forever. The lesson: consuming always means *advance the cursor*, never merely inspect.

2.3 Parsing expressions: the precedence cascade

Now translate the expression ladder from section 2.1 into code, one function per rule. Every function has the same shape, which is the payoff of writing the grammar first: parse one operand of the next level down, then loop while you see your operators, combining left-to-right. Here is the whole cascade for the binary levels:

```
def expression(self):
    return self.equality()

def equality(self):
    expr = self.comparison()
    while self.peek().kind in ('EQUAL_EQUAL', 'BANG_EQUAL'):
        op = self.advance()
        right = self.comparison()
        expr = Binary(op.lexeme, expr, right, op.line)
    return expr

def comparison(self):
    expr = self.term()
    while self.peek().kind in ('LESS','GREATER','LESS_EQUAL','GREATER_EQUAL'):
        op = self.advance()
        right = self.term()
        expr = Binary(op.lexeme, expr, right, op.line)
    return expr

def term(self):
    expr = self.factor()
    while self.peek().kind in ('PLUS', 'MINUS'):
        op = self.advance()
        right = self.factor()
        expr = Binary(op.lexeme, expr, right, op.line)
    return expr

def factor(self):
    expr = self.unary()
    while self.peek().kind in ('STAR', 'SLASH'):
        op = self.advance()
        right = self.unary()
        expr = Binary(op.lexeme, expr, right, op.line)
```

```
    return expr
```

Every one of these is the same seven lines with different operator sets and a different next-level call. That repetition is not a smell; it is the grammar made literal. The `expr = Binary(...)` line, which folds the running result back into the left operand, is the left-associativity you proved in Exercise 5.

The bottom two rules break the pattern slightly, because unary is prefix (recurse rightward) and primary is where the recursion bottoms out:

```
def unary(self):
    if self.peek().kind == 'MINUS':
        op = self.advance()
        operand = self.unary()          # note: recurse, allows --x
        return Unary(op.lexeme, operand, op.line)
    return self.primary()

def primary(self):
    tok = self.peek()
    if tok.kind in ('INT', 'FLOAT', 'STRING'):
        self.advance()
        return Literal(tok.value, tok.line)
    if tok.kind == 'IDENTIFIER':
        self.advance()
        return Variable(tok.lexeme, tok.line)
    if tok.kind == 'LEFT_PAREN':
        self.advance()
        expr = self.expression()        # back to the top of the ladder!
        self.consume('RIGHT_PAREN', "expected ')' after expression")
        return expr
    raise SyntaxError(f"Line {tok.line}: expected an expression")
```

Two beautiful things happen in `primary`. First, the parenthesis case calls `self.expression()` — it jumps all the way back to the *top* of the ladder. That is how a parenthesised group can contain a whole fresh expression of any complexity, and how the recursion in "recursive descent" closes its loop. Second, notice that the parentheses are `consume`d but never turned into a node — the returned `expr` is just the inner expression. This is section 1.2 in action: the parens shaped the tree and then disappeared.

> **Exercise 7 — Hand-trace the cascade (paper)**
>
> Trace, step by step, how the cascade parses the tokens for `1 + 2 * 3`. Write down, in order, which function is entered, what it parses, and what node it returns, until you reach the final tree. State the final tree in parenthesised form.
>
> This is the single most important exercise in the manual. Do not skip it.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 7

Following the calls (indentation shows the call depth):

```
expression()
  equality()  -&gt; no ==/!= ahead, so it is just...
    comparison() -&gt; no comparison ops, so just...
      term()
        factor()
          unary()
            primary() sees INT 1 -&gt; returns Literal(1)
          factor: next is '+', not */ -&gt; loop skipped
          factor returns Literal(1)
        term: first factor = Literal(1). Next is '+', a term op!
          advance past '+'
          call factor() for the right operand:
            factor()
              unary()-&gt;primary() sees INT 2 -&gt; Literal(2)
              factor: next is '*', a factor op! loop runs:
                advance past '*'
                unary()-&gt;primary() sees INT 3 -&gt; Literal(3)
                combine: Binary('*', Literal(2), Literal(3))
              factor returns Binary('*', 2, 3)
          term combines: Binary('+', Literal(1), Binary('*',2,3))
        term: next is EOF-ish, loop ends
      returns up the chain unchanged
```

Final tree: `(+ 1 (* 2 3))`. The key moment is that when `term` asked for its right operand, it called `factor`, and `factor` greedily swallowed the `2 * 3` as a single unit *before returning*. That greed — a tighter level consuming all of its operators before a looser level ever sees them — is precedence, mechanically realised.

2.4 Parsing statements

Statements are easier than expressions because there is no precedence to manage; you just look at the first token and decide which kind of statement you are looking at. This is a dispatch, much like the first-character family dispatch in your lexer — here it is a first-token dispatch.

```
def statement(self):
    kind = self.peek().kind
    if kind == 'PRINT':  return self.print_stmt()
    if kind == 'LET':    return self.let_stmt()
    if kind == 'IF':     return self.if_stmt()
    if kind == 'LEFT_BRACE': return self.block()
    return self.expr_stmt()

def print_stmt(self):
    kw = self.advance()                       # 'print'
    self.consume('LEFT_PAREN', "expected '(' after print")
    expr = self.expression()
    self.consume('RIGHT_PAREN', "expected ')' after print value")
    self.consume('SEMICOLON', "expected ';' after print statement")
    return PrintStmt(expr, kw.line)

def let_stmt(self):
    kw = self.advance()                       # 'let'
    name = self.consume('IDENTIFIER', "expected a name after 'let'")
    self.consume('EQUAL', "expected '=' in let declaration")
    value = self.expression()
    self.consume('SEMICOLON', "expected ';' after let declaration")
```

```
    return LetStmt(name.lexeme, value, kw.line)
```

The `if` statement and the block reuse `statement` and each other, which is where the recursion returns for statements:

```
def block(self):
    brace = self.consume('LEFT_BRACE', "expected '{'")
    stmts = []
    while not self.check('RIGHT_BRACE') and not self.check('EOF'):
        stmts.append(self.statement())
    self.consume('RIGHT_BRACE', "expected '}' to close block")
    return Block(stmts, brace.line)

def if_stmt(self):
    kw = self.advance()                       # 'if'
    self.consume('LEFT_PAREN', "expected '(' after if")
    cond = self.expression()
    self.consume('RIGHT_PAREN', "expected ')' after condition")
    then_branch = self.block()
    else_branch = None
    if self.match('ELSE'):
        else_branch = self.block()
    return IfStmt(cond, then_branch, else_branch, kw.line)
```

And the top-level driver ties it together: keep parsing statements until the `EOF` token that your lexer helpfully appended.

```
def parse(self):
    statements = []
    while not self.check('EOF'):
        statements.append(self.statement())
    return statements        # the whole program, a list of stmt nodes
```

Exercise 8 — Write a parse function from the grammar (paper)

The grammar rule for a hypothetical `while` statement (a stretch goal you might add) would be:

```
whileStmt -&gt; "while" "(" expression ")" block ;
```

Write the `while_stmt(self)` method that parses it and returns a `WhileStmt(condition, body, line)` node, using only the helpers and patterns shown so far. You do not need to define the dataclass; assume it exists.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 8

It is a near-copy of `if_stmt` without the else — which is exactly the point: once you have the pattern, new statement forms are mechanical.

```
def while_stmt(self):
    kw = self.advance()                    # 'while'
    self.consume('LEFT_PAREN', "expected '(' after while")
    cond = self.expression()
    self.consume('RIGHT_PAREN', "expected ')' after condition")
    body = self.block()
    return WhileStmt(cond, body, kw.line)
```

You would then add `if kind == 'WHILE': return self.while_stmt()` to the dispatch in `statement`. Note how the required punctuation uses `consume` (fail fast) while the structure — the condition and body — is delegated to the existing `expression` and `block` functions. Reuse, not repetition.

2.5 Errors, and knowing when to stop

Your fail-fast choice removes an entire category of difficulty from the parser. Serious language implementations invest heavily in *error recovery* — after a syntax error, skipping tokens until they reach a likely-safe point (usually the next semicolon) so they can report several errors in one run. You have explicitly, and wisely, chosen not to do this. The first `SyntaxError` that a `consume` or a `primary` fall-through raises will propagate all the way out and stop the program. One clear error with a line number is a perfectly good behaviour for a language of this size, and it is defensible in your writeup as a deliberate scope decision, not a gap.

Exercise 9 — Predict the error (paper)

For each malformed program, name which parser method raises the error, and what message and line it reports. Assume everything is on the line shown.

```
(a)  Line 4:   let 3 = x;
(b)  Line 9:   print(2 + );
(c)  Line 2:   let y = 5
```

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 9

(a) `let_stmt` advances past `let`, then calls `consume('IDENTIFIER', ...)`. The next token is `INT 3`, not an identifier, so consume fails: `Line 4: expected a name after 'let'`.

(b) `print_stmt` parses the expression, which reaches `term`; after the `+` it calls `factor` for the right operand, which descends to `primary`. But the next token is `)`, which matches none of primary's cases, so primary's final line raises `Line 9: expected an expression`.

(c) `let_stmt` parses everything up to the value `5`, then calls `consume('SEMICOLON', ...)`. The next token is `EOF` (or the start of the next line), so consume fails: `Line 2: expected ';' after let declaration`. This is your semicolons-as-terminators rule doing its job.

> **Milestone checkpoint**
>
> If you implement everything up to here — the helpers, the expression cascade, and statement parsing — you can already write a small *tree-walking interpreter* that evaluates the AST directly, before you ever touch bytecode. That is an excellent intermediate milestone: it proves your parser end to end, and it gives you a reference to check the VM against later. A tree-walker for expressions is about fifteen lines: the same `isinstance` walk as `show`, but returning computed values instead of strings.

PART 3

Opcodes and Bytecode

Designing the tiny instruction set your VM will run.

You now have a tree. You could stop and walk it directly to run programs (the tree-walker just mentioned). But your project's design — following the bytecode half of *Crafting Interpreters* — goes one step further: compile the tree down to **bytecode**, a flat list of simple numeric instructions, and run those on a **virtual machine**. This part designs the instructions; Part 4 emits them; Part 5 runs them.

3.1 What is a stack machine?

Your VM will be a *stack machine*, and this one idea makes the whole bytecode approach simple. Imagine a single stack of values — a pile you can only push onto the top of, or pop off the top of. Every instruction does its work by taking its inputs off the top of the stack and pushing its result back. There are no variables-as-registers to juggle; the stack is the scratch space.

The concrete image to hold is the spring-loaded plate dispenser you see in a canteen: you can only add a plate to the top, and you can only take the top plate off — no reaching into the middle. That single restriction is what makes the instructions interact so cleanly. Because everything happens at the top, an instruction never has to say *where* its inputs are; they are always the top one or two plates, by definition. So the instructions communicate with each other *entirely through the stack*, never directly: one instruction leaves a value on top as its output, and the very next instruction that needs an input finds it sitting right there. The stack is the shared workbench that every instruction reads from and writes to, and that is the only channel between them — which is why you can reason about each opcode in isolation, knowing only "what is on top when I start" and "what I leave on top when I finish."

Consider adding 2 and 3. In a stack machine that is three instructions: "push 2," "push 3," "add." The add instruction does not carry its operands; it *pops the top two values, adds them, and pushes the sum*. Written as a running picture of the stack, top of stack on the right:

```
instruction        stack after it runs
-----------        -------------------
PUSH 2             [ 2 ]
PUSH 3             [ 2, 3 ]
ADD                [ 5 ]        (popped 2 and 3, pushed 5)
```

This is why the compiler emits code in *post-order*: to add two sub-expressions, you first emit the code that leaves the left value on the stack, then the code that leaves the right value on the stack, then a single ADD. The nesting of the tree becomes the ordering of pushes, and the operators come after their operands. A stack machine is, in a real sense, the flat form of post-order tree traversal — which is why compiling to it is short.

3.2 The Plinkus instruction set

An opcode is just a number that names an operation. In Python the readable way to define them is a set of named integer constants. Here is a complete instruction set for your feature set. Some instructions take an *operand* (an extra number that follows the opcode in the byte stream); most do not.

```
# opcodes.py  -- each is just a distinct integer
OP_CONSTANT      = 0   # operand: index into constant pool; push that value
OP_ADD           = 1   # pop b, pop a, push a+b
OP_SUB           = 2
OP_MUL           = 3
```

```
OP_DIV           = 4   # always produces a float, per your rule
OP_NEGATE        = 5   # pop a, push -a
OP_EQUAL         = 6   # pop b, pop a, push (a == b)
OP_NOT_EQUAL     = 7
OP_LESS          = 8
OP_LESS_EQUAL    = 9
OP_GREATER       = 10
OP_GREATER_EQUAL = 11
OP_PRINT         = 12  # pop a, print it
OP_POP           = 13  # pop a, discard it (for expression statements)
OP_DEFINE_GLOBAL = 14  # operand: name index; pop value, bind name=value
OP_GET_GLOBAL    = 15  # operand: name index; push value bound to name
OP_SET_GLOBAL    = 16  # operand: name index; assign (stretch: reassignment)
OP_JUMP          = 17  # operand: offset; move instruction pointer forward
OP_JUMP_IF_FALSE = 18  # operand: offset; pop cond, jump if it is false
OP_RETURN        = 19  # end of program / return
```

Two things are worth dwelling on. First, **the constant pool.** You cannot store an arbitrary number or string *inside* a single-byte instruction, so instead you keep a side list of literal values — the constant pool — and `OP_CONSTANT` carries an *index* into it. To compile `3.14` you append `3.14` to the pool (say it lands at index 5) and emit `OP_CONSTANT 5`. Strings and the names of globals live in pools the same way.

The index is doing exactly what a coat-check ticket does, and the analogy makes the interaction between the two halves clear. The bulky coat (your `3.14`, or a long string) is too big to carry around, so you hand it to the cloakroom — the constant pool — and keep only a small numbered ticket, the index. The instruction stream carries the ticket, not the coat. Later, when the VM runs `OP_CONSTANT 5`, it reads ticket number 5, walks to the cloakroom (`chunk.constants[5]`), collects the actual value, and pushes *that* onto the stack. So the pool and the code stream are two cooperating structures: the code says *which* constant by number, and the pool holds the constant itself — and the index is the only thing linking them. Get the two out of step (append to the pool but emit the wrong index, or vice versa) and the VM will cheerfully push the wrong coat, which is a classic source of "why is my program using a number I never wrote?" bugs.

Second, **comparisons and arithmetic carry no operand** — they get everything they need from the stack. Only the instructions that must refer to something external (a constant, a name, a jump distance) carry an operand. This keeps the byte stream simple.

3.3 What a chunk looks like in Python

A compiled program is called a *chunk*. For Plinkus, the simplest representation that will not fight you is: a Python list of integers for the code, a parallel list of source lines (for runtime error messages), and the constant and name pools. A small class holds them together:

```
class Chunk:
    def __init__(self):
        self.code = []        # list of ints: opcodes and their operands
        self.lines = []       # code[i] came from source line lines[i]
        self.constants = []   # literal values; OP_CONSTANT indexes this
        self.names = []       # global names; GLOBAL ops index this

    def write(self, byte, line):
        self.code.append(byte)
        self.lines.append(line)
        return len(self.code) - 1        # position, used for backpatching

    def add_constant(self, value):
        self.constants.append(value)
        return len(self.constants) - 1   # its index in the pool
```

So an instruction with an operand occupies *two* slots in `code`: the opcode integer, then the operand integer. An instruction with no operand occupies one. The VM, when it reads an opcode, knows from the opcode itself whether to read one more slot for an operand. That "read the opcode, then read as many operand slots as this opcode needs" step is called *decoding*.

Exercise 10 — Compile by hand (paper)

Write out, by hand, the exact sequence of chunk entries (opcodes and operands, in order) that should be produced for this statement. Assume the constant pool starts empty and you add constants as you meet them, left to right.

```
print(1 + 2 * 3);
```

Give the `constants` list and the `code` list (as a flat list of the values that go into it, with a comment on each opcode).

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 10

Post-order: leaves pushed first, operators after their operands, print last. The multiplication is deeper, so it is emitted before the addition. Constants are added in the order the literals are first compiled: 1, then 2, then 3.

```
constants = [1, 2, 3]           # indices 0, 1, 2

code (flat, with meaning):
  OP_CONSTANT, 0    ; push 1        (index 0 -&gt; value 1)
  OP_CONSTANT, 1    ; push 2
  OP_CONSTANT, 2    ; push 3
  OP_MUL            ; pop 3,2 -&gt; push 6
  OP_ADD            ; pop 6,1 -&gt; push 7
  OP_PRINT          ; pop 7 -&gt; print 7
```

Trace the stack to check: push 1 -> [1]; push 2 -> [1,2]; push 3 -> [1,2,3]; MUL -> [1,6]; ADD -> [7]; PRINT -> [] and “7” appears on screen. The stack is empty at the end, which is the signature of a correctly balanced statement — every value produced was consumed.

PART 4

The Compiler

Walking the tree to emit the bytecode.

The compiler is a tree walk — the same recursive shape as your `show` printer, but instead of building a string it *emits instructions into a chunk*. Because the stack machine wants post-order, the rule for every expression node is: emit the children first, then emit the operator. That single discipline makes the compiler almost mechanical.

4.1 Compiling expressions

```
class Compiler:
    def __init__(self):
        self.chunk = Chunk()

    def emit(self, byte, line):
        return self.chunk.write(byte, line)

    def compile_expr(self, node):
        if isinstance(node, Literal):
            idx = self.chunk.add_constant(node.value)
            self.emit(OP_CONSTANT, node.line)
            self.emit(idx, node.line)

        elif isinstance(node, Variable):
            idx = self.name_index(node.name)
            self.emit(OP_GET_GLOBAL, node.line)
            self.emit(idx, node.line)

        elif isinstance(node, Unary):
            self.compile_expr(node.operand)   # child first
            self.emit(OP_NEGATE, node.line)    # then the operator

        elif isinstance(node, Binary):
            self.compile_expr(node.left)       # left child
            self.compile_expr(node.right)      # right child
            self.emit(BINARY_OPS[node.op], node.line)  # then operator

        else:
            raise Exception(f"cannot compile expr {node!r}")
```

The `Binary` case is the heart of it. Compile the left subtree (which leaves its value on the stack), compile the right subtree (which leaves its value above it), then emit one operator instruction that pops both and pushes the result. `BINARY_OPS` is just a dict mapping the operator string to its opcode:

```
BINARY_OPS = {
    '+': OP_ADD, '-': OP_SUB, '*': OP_MUL, '/': OP_DIV,
    '==': OP_EQUAL, '!=': OP_NOT_EQUAL,
    '&lt;': OP_LESS, '&lt;=': OP_LESS_EQUAL,
    '&gt;': OP_GREATER, '&gt;=': OP_GREATER_EQUAL,
}
```

4.2 Compiling statements and globals

Statements compile their sub-expression(s) and then emit the action. A `print` compiles its expression (leaving the value on the stack) then emits `OP_PRINT`. A `let` compiles its value expression then emits

`OP_DEFINE_GLOBAL` with the name's index — the VM will pop the value and bind the name to it.

```
def compile_stmt(self, node):
    if isinstance(node, PrintStmt):
        self.compile_expr(node.expr)
        self.emit(OP_PRINT, node.line)

    elif isinstance(node, LetStmt):
        self.compile_expr(node.value)        # value now on stack
        idx = self.name_index(node.name)
        self.emit(OP_DEFINE_GLOBAL, node.line)
        self.emit(idx, node.line)

    elif isinstance(node, ExprStmt):
        self.compile_expr(node.expr)
        self.emit(OP_POP, node.line)         # discard the unused value

    elif isinstance(node, Block):
        for s in node.statements:
            self.compile_stmt(s)

    elif isinstance(node, IfStmt):
        self.compile_if(node)                # see 4.3
    else:
        raise Exception(f"cannot compile stmt {node!r}")
```

The `ExprStmt` case explains why `OP_POP` exists: an expression statement computes a value nobody wants, so you must pop it to keep the stack balanced. This “every statement leaves the stack as it found it” rule is the invariant that keeps a stack machine from slowly leaking values.

4.3 If-statements: jumps and backpatching

Conditionals are the one genuinely tricky thing in the compiler, and the difficulty is this: when you emit the jump that skips the `then` branch, you do not yet know how far to jump, because you have not compiled the branch yet, so you do not know how long it is. The solution is **backpatching**: emit the jump with a placeholder operand, remember where the placeholder is, compile the branch, and then go back and overwrite the placeholder with the now-known distance.

Think of it as writing “go forward ___ steps” in pencil, walking the path to count the steps, and then coming back to fill in the blank. Here is the logic for `if` with an optional `else`:

```
def compile_if(self, node):
    self.compile_expr(node.condition)       # condition value on stack

    # emit jump-if-false with a placeholder distance:
    self.emit(OP_JUMP_IF_FALSE, node.line)
    jif = self.emit(0xFFFF, node.line)      # placeholder operand slot

    self.compile_stmt(node.then_branch)     # the 'then' code

    if node.else_branch is not None:
        # at end of 'then', jump over the 'else':
        self.emit(OP_JUMP, node.line)
        jelse = self.emit(0xFFFF, node.line)
        self.patch(jif)                     # false lands here (start of else)
        self.compile_stmt(node.else_branch)
        self.patch(jelse)                   # then-jump lands here (after else)
    else:
        self.patch(jif)                     # false lands here (after then)

def patch(self, slot):
```

```
    # distance from just-after-the-operand to the current end of code
    self.chunk.code[slot] = len(self.chunk.code) - (slot + 1)
```

Read the flow in the no-else case: evaluate the condition; if it is false, jump forward past the whole `then` block; otherwise fall straight through and run it. The `patch` call, made *after* the block is compiled, computes how many slots the block occupied and writes that into the placeholder. In the else case there are two jumps: one to skip the `then` when the condition is false, and one at the end of `then` to skip over the `else` when the condition was true.

Why note the position when writing?

This is why `Chunk.write` returns the index it just wrote to. That returned position is the "pencil mark" you keep so you can find the placeholder again in `patch`. Almost every backpatching bug is either patching the wrong slot or computing the off-by-one distance wrong — so trace it on paper once, carefully, before trusting the code.

Exercise 11 — Compile an if by hand (paper)

Write the full chunk (code list with comments, and constants list) for the program below. Use the exact opcodes from Part 3. Show the placeholder and then the final patched distance for the jump. Assume `x` is name index 0 in the names pool.

```
if (x &gt; 5) {
    print(x);
}
```

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 11

Condition first (`x > 5`), then the conditional jump with a placeholder, then the then-branch (`print(x)`), then patch. Writing code positions on the left so we can compute the jump:

```
constants = [5]        ; index 0
names     = ['x']      ; index 0

pos  code                     ; meaning
---  ------------------------ ------------------------------
 0   OP_GET_GLOBAL            ; push value of x
 1     0                      ;   name index 0
 2   OP_CONSTANT             ; push 5
 3     0                      ;   const index 0
 4   OP_GREATER              ; pop 5,x -&gt; push (x &gt; 5)
 5   OP_JUMP_IF_FALSE        ; if false, jump over the then-block
 6     &lt;placeholder&gt;          ;   patched below
 7   OP_GET_GLOBAL            ; push value of x
 8     0
 9   OP_PRINT                ; pop and print x
10   (end)

Patch: the operand at pos 6 = (end 10) - (6 + 1) = 3.
So a false condition jumps 3 slots forward, from just after
pos 6 (i.e. pos 7) to pos 10 -- exactly past the then-block.
```

Check the jump arithmetic against the definition of `patch`: distance = current end minus (slot + 1). The slot is pos 6, current end is 10, so 10 - 7 = 3. When the VM reads a false condition, it will add 3 to its instruction pointer and land at pos 10, skipping the print. If the condition is true, it ignores the jump and falls through to pos 7.

PART 5

The Virtual Machine

The loop that runs the bytecode. The finish line.

The VM is deceptively simple: a loop that reads one instruction, does what it says, and repeats. It is called a *virtual* machine because it simulates a processor in software — it does not translate your bytecode into real CPU instructions; the Python interpreter runs the VM, and the VM interprets your bytecode. Two layers of interpretation, which is why bytecode VMs are slower than native code but far easier to write.

5.1 The fetch–decode–execute loop

The VM holds three pieces of state: the chunk it is running, an *instruction pointer* (`ip`) that indexes into `chunk.code`, and the value `stack`. The loop **fetches** the opcode at `ip`, **decodes** it (reads any operand slots it needs), and **executes** it.

The instruction pointer is your finger tracking your place as you follow a numbered list of instructions, and understanding how it interacts with `read_byte` is the key to the whole loop. `read_byte` does two things in one motion: it reads the slot the finger is on, *and* it moves the finger one slot forward. So reading is never passive — every read advances. This is what makes decoding operands work without any bookkeeping: after `read_byte` fetches an opcode like `OP_CONSTANT`, the finger is already resting on the operand slot, so a second `read_byte` picks up the index and advances again, leaving the finger on the *next* instruction, perfectly positioned for the top of the loop. The opcode and its operand thus get consumed together, in order, and the finger always ends an instruction sitting on the start of the following one. Jumps are simply the one place where you *pick the finger up and move it* by hand (`ip += offset`) instead of letting `read_byte` nudge it forward one step at a time — which is exactly how a conditional skips over a block of code.

```
class VM:
    def __init__(self, chunk):
        self.chunk = chunk
        self.ip = 0
        self.stack = []
        self.globals = {}

    def read_byte(self):
        b = self.chunk.code[self.ip]
        self.ip += 1
        return b

    def run(self):
        code = self.chunk.code
        while self.ip &lt; len(code):
            op = self.read_byte()               # FETCH
            if op == OP_CONSTANT:               # DECODE + EXECUTE
                idx = self.read_byte()          # operand
                self.stack.append(self.chunk.constants[idx])
            elif op == OP_ADD:
                b = self.stack.pop(); a = self.stack.pop()
                self.stack.append(self.binary_add(a, b))
            elif op == OP_SUB:
                b = self.stack.pop(); a = self.stack.pop()
                self.stack.append(a - b)
            elif op == OP_MUL:
                b = self.stack.pop(); a = self.stack.pop()
                self.stack.append(a * b)
            elif op == OP_DIV:
```

```
            b = self.stack.pop(); a = self.stack.pop()
            self.stack.append(a / b)        # Python / is always float
        elif op == OP_NEGATE:
            self.stack.append(-self.stack.pop())
        elif op == OP_PRINT:
            print(self.stack.pop())
        elif op == OP_POP:
            self.stack.pop()
        # ... comparisons, globals, jumps: below ...
        elif op == OP_RETURN:
            return
```

Notice how `OP_ADD` pops `b` *first* and then `a`. Order matters: the compiler pushed the left operand before the right, so the right operand (`b`) is on top and comes off first. For commutative `+` it would not matter, but for `-` and `/` it absolutely does — getting this backwards makes `10 - 3` compute as `3 - 10`. Always pop right, then left.

5.2 Runtime types: promotion and division

Your two type rules land here, in the VM, because types are a runtime property of the values on the stack. Integer-to-float promotion on mixed arithmetic is mostly free: Python already promotes `2 + 3.0` to `5.0`. But you should make the rule explicit rather than leaning on Python's coincidental behaviour, both so it is correct for string concatenation and so your writeup can describe the semantics precisely:

```
def binary_add(self, a, b):
    if isinstance(a, str) and isinstance(b, str):
        return a + b                    # string concatenation
    if isinstance(a, (int, float)) and isinstance(b, (int, float)):
        return a + b                    # int+float promotes to float
    raise RuntimeError(
        f"Line {self.current_line()}: cannot add "
        f"{type(a).__name__} and {type(b).__name__}")
```

Your rule that `/` always yields a float is, happily, exactly what Python's `/` operator does, so `OP_DIV` can simply use it. If you ever wanted integer division you would use `//`, but you have decided against that, so `/` is enough. To report the line in a runtime error, keep a parallel index into `chunk.lines` as you execute — that is what the `lines` list on the chunk was for.

5.3 Globals and jumps

Globals use the `globals` dictionary keyed by name. `OP_DEFINE_GLOBAL` reads its name index, looks up the actual name string in `chunk.names`, pops the value, and stores the binding. `OP_GET_GLOBAL` does the reverse, and this is where an “undefined variable” runtime error lives:

```
        elif op == OP_DEFINE_GLOBAL:
            name = self.chunk.names[self.read_byte()]
            self.globals[name] = self.stack.pop()
        elif op == OP_GET_GLOBAL:
            name = self.chunk.names[self.read_byte()]
            if name not in self.globals:
                raise RuntimeError(
                    f"Line {self.current_line()}: "
                    f"undefined variable '{name}'")
            self.stack.append(self.globals[name])
```

Jumps simply move the instruction pointer. `OP_JUMP` reads its offset and adds it to `ip`. `OP_JUMP_IF_FALSE` reads its offset, then pops the condition value; only if that value is false does it add

the offset — otherwise it falls through, having already advanced past the offset slot when it read it.

```
elif op == OP_JUMP:
    offset = self.read_byte()
    self.ip += offset
elif op == OP_JUMP_IF_FALSE:
    offset = self.read_byte()
    condition = self.stack.pop()
    if not self.is_truthy(condition):
        self.ip += offset
```

The distances patched in during compilation (Exercise 11) were measured precisely so this `self.ip += offset` lands on the right instruction. Because `read_byte` already advanced `ip` past both the opcode and the offset slot, the offset is counted from that point — which is exactly the convention your `patch` used (`slot + 1`). The two must agree; that agreement is the whole game with jumps.

Exercise 12 — Run the VM in your head (paper)

Given the chunk below (constants `[10, 4]`), trace the VM: show the stack after every instruction, and state what, if anything, is printed.

```
OP_CONSTANT 0     ; 10
OP_CONSTANT 1     ; 4
OP_SUB
OP_CONSTANT 1     ; 4
OP_DIV
OP_PRINT
```

Be careful with pop order on `OP_SUB` and `OP_DIV`.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 12

```
instruction        stack after       note
OP_CONSTANT 0      [10]
OP_CONSTANT 1      [10, 4]
OP_SUB             [6]               pop b=4, a=10 -&gt; 10-4 = 6
OP_CONSTANT 1      [6, 4]
OP_DIV             [1.5]             pop b=4, a=6 -&gt; 6/4 = 1.5 (float!)
OP_PRINT           []                prints: 1.5
```

It prints `1.5`. Two things confirm your rules: the subtraction popped right-then-left so it computed `10 - 4` and not `4 - 10`; and the division produced `1.5`, a float, even though both operands were integers — your "`/` is always float" rule, delivered for free by Python's `/`.

5.4 A full end-to-end trace

Put the whole pipeline together on one tiny program. This is the trace to reproduce on the plane, because if you can follow it from source to output you understand every stage of your project.

```
SOURCE:
  let x = 2;
  print(x + 1);

AST (indented):
  LetStmt name='x' value=Literal 2
  PrintStmt expr=Binary '+'
                   left=Variable 'x'
                   right=Literal 1

CHUNK (constants=[2, 1], names=['x']):
  0  OP_CONSTANT 0      ; push 2
  2  OP_DEFINE_GLOBAL 0 ; pop 2, globals['x'] = 2
  4  OP_GET_GLOBAL 0    ; push globals['x'] = 2
  6  OP_CONSTANT 1      ; push 1
  8  OP_ADD             ; pop 1,2 -&gt; push 3
  9  OP_PRINT           ; pop 3 -&gt; print 3

VM RUN (stack shown after each):
  OP_CONSTANT 0       [2]
  OP_DEFINE_GLOBAL 0  []          globals = {'x': 2}
  OP_GET_GLOBAL 0     [2]
  OP_CONSTANT 1       [2, 1]
  OP_ADD              [3]
  OP_PRINT            []          OUTPUT: 3
```

Exercise 13 — The whole pipeline, once, by yourself (paper)

Do for this program what section 5.4 did for its example: write the AST (indented), the chunk (with constants and names pools and code positions), and the VM stack trace, ending with the output. Assume `if` here has no else branch.

```
let n = 4;
if (n &lt; 5) {
    print(n * 2);
}
```

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 13

AST (indented):

```
LetStmt name='n' value=Literal 4
IfStmt
    condition: Binary '&lt;'
                   left=Variable 'n'  right=Literal 5
    then_branch: Block
        PrintStmt expr=Binary '*'
                         left=Variable 'n'  right=Literal 2
    else_branch: None
```

CHUNK (constants=[4, 5, 2], names=['n']):

```
pos code                    ; meaning
 0  OP_CONSTANT 0           ; push 4
 2  OP_DEFINE_GLOBAL 0      ; globals['n'] = 4
 4  OP_GET_GLOBAL 0         ; push n
 6  OP_CONSTANT 1           ; push 5
 8  OP_LESS                 ; push (n &lt; 5)
 9  OP_JUMP_IF_FALSE        ; skip then-block if false
10    6                     ; patched distance = 17 - (10+1) = 6
11  OP_GET_GLOBAL 0         ; push n
13  OP_CONSTANT 2           ; push 2
15  OP_MUL                  ; push n*2
16  OP_PRINT                ; print it
17  (end)
```

VM run (stack after each):

```
OP_CONSTANT 0        [4]
OP_DEFINE_GLOBAL 0   []          globals={'n':4}
OP_GET_GLOBAL 0      [4]
OP_CONSTANT 1        [4, 5]
OP_LESS              [True]      4 &lt; 5 is true
OP_JUMP_IF_FALSE 6   []          cond True -&gt; do NOT jump, fall through
OP_GET_GLOBAL 0      [4]
OP_CONSTANT 2        [4, 2]
OP_MUL               [8]
OP_PRINT             []          OUTPUT: 8
```

The condition was true, so `OP_JUMP_IF_FALSE` popped `True` and fell through into the block. Had `n` been, say, 9, the comparison would push `False`, the jump would add 6 to `ip` (from pos 11 to pos 17), and nothing would print — which is precisely the behaviour an `if` should have.

5.5 Wiring the four stages together

You have now seen every stage on its own; this is how they clip together into one runnable whole. Each stage is a function that takes the previous stage's output as its input and produces the next stage's input — the same chain from Part 0, now in code you can actually call. Nothing here is new machinery; it is just the plumbing that connects the pieces:

```
def run_source(source):
    tokens = Lexer(source).scan_tokens()   # chars  -&gt; tokens   (done)
    ast    = Parser(tokens).parse()        # tokens -&gt; AST      (Part 2)
    comp   = Compiler()
    for stmt in ast:                       # AST    -&gt; bytecode (Part 4)
        comp.compile_stmt(stmt)
    comp.emit(OP_RETURN, 0)                # mark the end for the VM
```

```
VM(comp.chunk).run()                # bytecode runs      (Part 5)
```

Trace the hand-offs, because the interaction between the stages is the point. The lexer's *output* — the token list — is literally the argument you feed to the parser. The parser's output — the list of AST nodes — is what the compiler loops over. The compiler does not return bytecode separately; it fills in `comp.chunk` as a side effect, and that same chunk object is then handed straight to the VM's constructor. So the four stages form a bucket brigade: tokens flow into the parser, a tree flows into the compiler, a chunk flows into the VM, and output flows onto your screen. If any stage's output shape does not match what the next stage expects — say the parser returns a single node instead of a list — the seam between them is exactly where the error will surface, which is why building one thin slice all the way through (Part 6) is the fastest way to find a broken hand-off.

> **Why OP_RETURN at the end**
>
> The extra `OP_RETURN` the driver appends is the full-stop at the end of the byte stream. The VM's loop runs until it either runs off the end of the code or hits a `return`; emitting one explicitly gives the loop a clean, defined place to stop rather than relying on the list simply running out. It is the interaction between compiler and VM in miniature: the compiler leaves a signal, and the VM is written to recognise it.

PART 6

A Plan of Attack

Milestones, the bugs that will bite, and what to do first.

6.1 Build in the smallest runnable slices

Do not build the parser, then the whole compiler, then the whole VM. Build a thin vertical slice all the way through the pipeline, get it running, then widen it. Each slice below is a program you can actually execute, which means each is a place to catch bugs while the surface area is still small — the same incremental, verified approach that worked for your lexer.

- **Slice 1 — integer arithmetic.** Parser handles the expression cascade for `+ - * /` and literals; compiler emits `OP_CONSTANT` and the four arithmetic ops; VM runs them. Test target: a bare expression like `2 + 3 * 4` evaluated and printed gives 14.
- **Slice 2 — print and floats.** Add the `print` statement and float literals with promotion. Target: `print(7 / 2);` gives 3.5.
- **Slice 3 — variables.** Add `let`, `Variable`, and the global opcodes. Target: `let x = 10; print(x + 5);` gives 15.
- **Slice 4 — comparisons.** Add the six comparison operators through the `comparison` and `equality` rules and their opcodes. Target: `print(3 < 5);` gives True.
- **Slice 5 — if/else.** Add blocks, the `if` statement, jumps, and backpatching. This is the hardest slice; do it last, and trace it on paper before coding. Target: your two canonical example programs run correctly end to end.

By slice 5 you will have run each canonical example many times over smaller sub-features, so the final integration is far less likely to present you with a wall of simultaneous bugs.

6.2 The bugs that will actually bite you

- **Inverted operand order in the VM.** Popping left-then-right instead of right-then-left silently breaks `-` and `/` while leaving `+` and `*` looking fine. If subtraction is wrong but addition is right, this is your culprit.
- **Precedence inverted in the cascade.** If `2 + 3 * 4` gives 20, your `term` and `factor` are in the wrong order, or one of them calls the wrong next level. Run the output through your `show` printer to see the bad grouping directly.
- **Forgetting to advance.** A helper that checks a token but does not consume it produces infinite loops or repeated errors. If the parser hangs, look for a missing `advance`.
- **Off-by-one in jump distances.** The compiler's `patch` convention and the VM's `ip += offset` convention must count from the same point (just after the operand slot). If your `if` lands one instruction early or late, this is why.
- **Unbalanced stack.** If values pile up or the stack underflows, a statement is not cleaning up after itself — usually a missing `OP_POP` on an expression statement, or a `print` that forgot to consume its value.

6.3 Paper first, for two things

Your roadmap already flags two stages for paper design, and it is right. Design the **AST node catalogue** on paper by writing your two example programs and reading off, node by node, what types you need — you did this for tokens in the lexer and it worked. Design the **opcode set** on paper the same way: hand-compile both example programs into instructions (Exercises 10, 11, and 13 are rehearsals for exactly this) and let the opcodes you find yourself needing become the set. Design from the examples outward; do not invent instructions you never use.

Exercise 14 — Design your own milestone plan (paper)

You have two canonical example programs. Without looking at section 6.1, write down the order in which you will make them run — the smallest first runnable behaviour, then each widening step — and for each step, the one tiny test program that proves it works. Aim for four to six steps. Then compare against 6.1 and note any feature you would sequence differently and why.

→ *Attempt this now on paper (or in the Python interpreter). Turn the page when you are ready to check.*

Answer to Exercise 14

There is no single right answer — this is about having a plan — but a strong plan shares these properties, which are the things to check yours against:

- **Each step is independently runnable.** Every milestone ends with a program you can execute and eyeball, not an internal component with no visible behaviour.
- **Arithmetic comes first,** because it exercises the full parser-cascade-to-VM path with the fewest moving parts, so it is where you debug the pipeline itself.
- **Variables precede conditionals,** because your `if` examples reference variables, so you need globals working before you can meaningfully test a branch.
- **if/else is last,** because jumps and backpatching are the only genuinely new machinery and you want everything else solid underneath them first.
- **Each step names its test.** A milestone without a concrete "this program should print X" is not a milestone, it is a hope.

If your ordering put comparisons before variables, that is defensible — comparisons are pure expressions and need no globals — and arguably even cleaner, since it keeps all the pure-expression work together before you introduce state. The important thing is that you can justify the order, which is itself good material for your IA writeup.

6.4 Where this sits in your writeup

Every design decision in this manual is a paragraph you can defend in the IA. The stack machine over registers; bytecode over pure tree-walking; fail-fast over error recovery; dataclasses over a generated visitor; separate types with promotion; the precedence cascade as the mechanism for operator precedence — each is a choice with a stated reason and a named alternative. Assessors reward exactly that: not just that it works, but that you understood the space of options and chose deliberately for your scope. You have been building this way since the lexer; keep the same habit of writing down *why* as you go, while the reasoning is fresh, rather than reconstructing it at the end.

That is the whole road from here to a running Plinkus. Tree, parser, opcodes, compiler, VM — you have seen every stage now, in code you can type and traces you can reproduce. Close the manual, open the interpreter, and start with Slice 1. Land the plane with arithmetic already running.

End of Part II. When you have the VM running the two example programs, Part III would cover the stretch tier — loops via backward jumps, local variables and scope via stack slots, and functions via call frames — but none of that is needed for a complete, defensible core.